SWIFTSHIP TRACKER

Salesforce Implementation Project Documentation

Program: Salesforce Developer

Team Leader: Balamurugan S

Team Members:

BALAGANESH P	820323205025
ARUN PRASATH F	820323205020
ABISHEK V	820323205007
KIRESH R	820323205059

Registered Mail ID: balasabasrish098@gmail.com

Team number: 5

College: AVC College of Engineering

Platform: Salesforce Developer Edition Org (Lightning Experience)

Submission Date: 6th October 2026

SWIFTSHIP TRACKER

SALESFORCE IMPLEMENTATION

User Story

SwiftShip Tracker is a centralized Salesforce solution for handling parcel bookings, delivery tracking, and customer communication. Customers who book a parcel want to see where it is without phoning support or switching between several applications. Delivery agents want one place to update the status of the parcels they carry. The system stores Parcel, Delivery, Sender, and Receiver records in connected custom objects. A Prompt Builder template and an Autolaunched Flow feed an Agentforce agent that answers tracking questions in plain language, while permission sets, roles, and profiles decide who may see or change each record. Reports and dashboards give managers a view of parcel status and delivery performance.

1. INTRODUCTION

1.1 Project Overview

SwiftShip Tracker is a Salesforce CRM application designed for a parcel delivery business that needs to manage the whole parcel lifecycle in one platform: booking, dispatch, in-transit tracking, out-for-delivery, and delivery confirmation. Customers can look up their shipments, and delivery agents can maintain parcel and delivery details.

The data model has four custom objects: Parcel, Delivery, Sender, and Receiver. Automation is built with Salesforce Flow. The most distinctive part of the solution is its conversational layer: an Agentforce agent named SwiftShip Tracker uses a Prompt Builder template to read a parcel record and reply with its ID, status, weight, and estimated delivery date. The project was configured and tested in a Salesforce Developer Edition org for academic purposes.

1.2 Purpose

The system exists to make parcel booking and tracking simpler, faster, and more transparent. Specifically, it is designed to:

- Keep parcel, delivery, sender, and receiver information in one place.
- Make parcel booking and tracking straightforward for customers.
- Give real-time visibility of shipment status and location.
- Automate delivery updates and customer notifications.
- Allow customers to track parcels conversationally through Agentforce and Prompt Builder.
- Improve agent efficiency through Flows and dashboards.

- Provide secure, role-based access for customers, agents, support staff, and administrators.

1.3 Objectives and Business Value

The main objective is to digitize parcel operations and give both customers and staff a single reliable view of each shipment. The business value comes from the following:

- Less manual work: status updates and notifications run through automation instead of phone calls and spreadsheets.
- Better customer experience: customers can check a parcel at any time through selfservice tracking.
- Faster agents: delivery agents update one record rather than several systems.
- Controlled access: sensitive sender and receiver details are visible only to the right users.
- Management visibility: reports and dashboards summarize parcel flow and delivery performance.

2. IDEATION PHASE

2.1 Problem Statement

The table below uses the customer problem statement template.

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	Customer / Sender	book a parcel easily	booking needs support or several applications	parcel operations are not centralized	frustrated
PS-2	Customer / Sender	track my parcel	there is no simple real-time tracking	shipment information is hard to reach	confused
PS-3	Delivery Agent	update parcel status	status updates are handled manually	operational tracking needs to be centralized	concerned
PS-4	Customer	receive delivery updates	notifications are not sent at every stage	communication has to be automated	uncertain
PS-5	Admin	monitor parcel operations	parcel flow and delivery performance are hard to monitor	operational data needs central reporting	concerned

2.2 Empathy Map Canvas

User persona: Customer / Sender.

Quadrant	Description
Think & Feel	Wants to book and track parcels without repeatedly contacting support.
Hear	Gets delivery information and status messages from the parcel service.
See	Parcel details, tracking status, estimated delivery date, and delivery information.

Say & Do	Books parcels, checks tracking information, and asks for delivery updates.
Pain	Delays, confusion, no easy tracking, and dependence on support or several apps.
Gain	Real-time visibility, self-service tracking, convenient booking, and timely
Quadrant	**Description**
	notifications.

2.3 Brainstorming and Idea Prioritization

Idea	Approach	Key Limitation
Manual registers	Record parcels and deliveries on paper	Slow, error-prone, no tracking for customers
Spreadsheet tracking	Maintain parcel lists in spreadsheets	Little automation and no selfservice
Standalone parcel portal	Build a dedicated tracking website	High development and maintenance effort
Salesforce CRM with Agentforce	Centralize parcel data and add an AI tracking agent	Needs careful configuration of data, flows, and prompts

Final selection: Salesforce CRM with Agentforce was selected because it offers centralized parcel management, automation through Flows, real-time tracking visibility, conversational AI assistance, secure role-based access, and built-in reports and dashboards.

3. REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

Stage	Journey Stage	What Happens
1	Parcel Booking	The customer provides parcel, sender, and receiver details.
2	Parcel Record Creation	A Parcel record with an auto-numbered Parcel ID is created.
3	Dispatch and Shipment	The parcel is dispatched and a Delivery record is linked to it.
4	In-Transit Tracking	The agent updates status and current location.
5	Out for Delivery	Status moves to Out for Delivery.
6	Delivery Confirmation	Status is set to Delivered.
7	Customer Notification	Email alerts inform the customer of progress.
8	Feedback / Support	The customer contacts support or gives feedback if needed.

3.2 Solution Requirements Functional Requirements

FR No	Requirement
FR-1	Parcel booking and record creation

FR-2	Parcel status tracking
FR-3	Weight management
FR No	**Requirement**
FR-4	Estimated delivery date management
FR-5	Sender information management
FR-6	Receiver information management
FR-7	Delivery and current-location tracking
FR-8	Agent parcel status updates
FR-9	Automated customer notifications
FR-10	AI-powered parcel tracking
FR-11	Reports and dashboards
FR-12	Customer access through Experience Cloud

Non-Functional Requirements

NFR	Description
Usability	Simple Salesforce and customer-facing interface
Security	Role-based and field-level access control
Reliability	Accurate parcel status and delivery information
Performance	Timely Flow and Agentforce responses
Availability	Cloud-based access through Salesforce
Scalability	Supports growing parcel and delivery volumes
Data Integrity	Controlled access and relationship-based parcel data
Maintainability	Flows, prompts, topics, and configuration can be updated easily

3.3 Data Flow Diagram

Main flow: Customer / Sender → Parcel booking → Parcel record → Sender and Receiver information → Delivery record → Parcel status update → Current location and estimated delivery → Automated notifications → Customer tracking or Agentforce agent → Reports and dashboards.

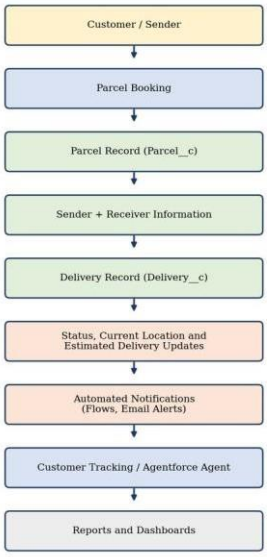


Figure 1: Data flow diagram of SwiftShip Tracker

AI tracking flow: the customer asks a question, the Agentforce agent passes the Parcel ID to a Flow, the Flow retrieves the Parcel record and calls the Prompt Builder template, and the template returns the Parcel ID, status, weight, and estimated delivery date as a conversational reply.

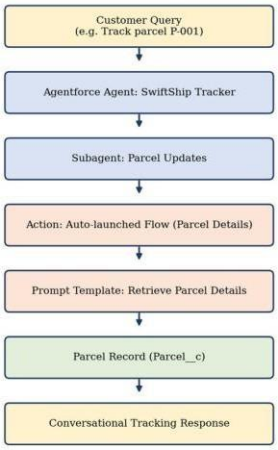


Figure 2: AI tracking flow

3.4 Technology Stack

Layer	Technology
Platform	Salesforce Developer Edition
User Interface	Lightning App, Lightning Record Pages, Experience Cloud
Database	Salesforce custom and standard objects
Business Logic	Flows, Apex classes
Automation	Record-triggered, auto-launched, and scheduled Flows
AI	Agentforce, Prompt Builder
Notifications	Email alerts and email templates
Layer	**Technology**

Processing	Batch Apex
Security	Profiles, permission sets, role hierarchy, field-level security, sharing rules
Reporting	Reports and dashboards

4. PROJECT DESIGN PHASE

4.1 Problem–Solution Fit

Problem	SwiftShip Solution	Benefit
Parcel booking is difficult	Parcel management on Salesforce	Centralized booking
Tracking depends on support	Agentforce agent with Prompt Builder	Self-service tracking
Status updates are manual	Salesforce Flows	Automated processing
Gaps in delivery communication	Email alerts	Timely notifications
Shipment data is scattered	Parcel, Delivery, Sender, Receiver objects	Structured data
Operations are hard to monitor	Reports and dashboards	Better visibility
Users need different access	Profiles, roles, permission sets	Secure access

4.2 Proposed Solution

Parameter	Description
Problem	Parcel booking and tracking is slow, scattered, and depends on support staff.
Solution	A Salesforce-based parcel management system.
Innovation	An Agentforce agent with Prompt Builder gives conversational selfservice tracking, supported by automated Flows.
Impact	Centralized booking, real-time tracking visibility, timely notifications, and efficient delivery management.
Business Model	Serves customers, delivery agents, support staff, and administrators of a parcel business.
Scalability	Can grow with parcel volume, Experience Cloud users, and integration with external logistics systems.

4.3 Solution Architecture

The solution is arranged in layers. Users reach the platform through the Lightning App, Experience Cloud, or the Agentforce chat. Below that sit the automation layer, the AI layer, the data model, and the security layer.

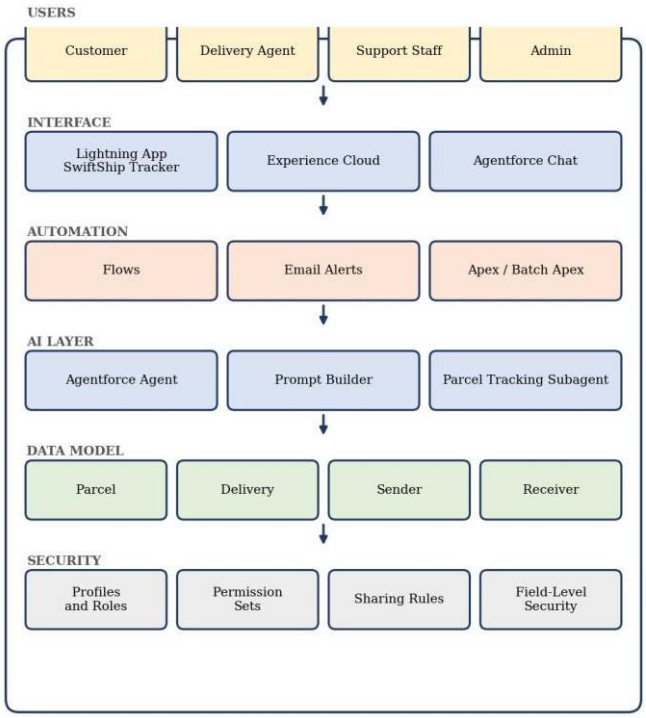


Figure 3: Solution architecture of SwiftShip Tracker

5. PROJECT PLANNING AND SCHEDULING

5.1 Project Planning

The project followed an Agile approach. Requirements were grouped into epics, split into user stories with story points, and delivered across six sprints. Sprint velocity was used to estimate the following sprint.

Product Backlog and Sprint Schedule

Sprint	Epic	US No	User Story / Task	Pts	Priority	Member
1	Developer Setup	USN-1	As a system administrator, I want to create and configure the Salesforce environment so that I can build the parcel management system.	3	High	Balamurugan S
2	Data Modeling	USN-2	As an admin, I want to create Parcel, Delivery, Sender, and Receiver objects with relationships so that parcel data is stored systematically.	5	High	Balaganesh P
2	Data Modeling	USN-3	As a staff member, I want tabs and a Lightning App for SwiftShip modules so that I can navigate parcel and delivery operations easily.	5	High	Arun Prasath F
3	Automation	USN-4	As a system, I want to validate parcel details and mandatory fields so that incomplete parcel	3	High	Abishek V
Sprint	**Epic**	**US No**	**User Story / Task**	**Pts**	**Priority**	**Member**

			records are prevented.			
3	Automation	USN-5	As a delivery coordinator, I want Flows to update parcel status and delivery information so that processing needs minimal manual effort.	3	High	Kiresh R
4	Apex	USN-6	As a system, I want Apex triggers and classes to perform critical parcel and delivery actions accurately.	5	High	Balamurugan S
4	Apex	USN-7	As an operations manager, I want Batch Apex jobs to update parcel and delivery records automatically so that data stays current.	5	High	Balaganesh P
5	Reports	USN-8	As a manager, I want reports on parcel status and delivery performance so that I can monitor operations.	4	High	Arun Prasath F
6	Dashboards and AI	USN-9	As a manager, I want dashboards and Agentforce tracking assistance so that I can monitor operations and customers can self-serve.	4	Medium	Kiresh R

Story ownership: Balamurugan S (team leader), Balaganesh P, Arun Prasath F, Abishek V and Kiresh R each owned the stories listed against their names.

Project Tracker, Velocity and Burndown

Sprint	Total Points	Duration	Start Date (Planned)	End Date (Planned)	Points Completed	Release Date (Actual)
Sprint-1	3	6 days	[dd Mon yyyy]	[dd Mon yyyy]		[dd Mon yyyy]
Sprint-2	10	6 days	[dd Mon yyyy]	[dd Mon yyyy]		[dd Mon yyyy]
Sprint-3	6	6 days	[dd Mon yyyy]	[dd Mon yyyy]		[dd Mon yyyy]
Sprint-4	10	6 days	[dd Mon yyyy]	[dd Mon yyyy]		[dd Mon yyyy]
Sprint-5	4	6 days	[dd Mon yyyy]	[dd Mon yyyy]		[dd Mon yyyy]
Sprint-6	4	6 days	[dd Mon yyyy]	[dd Mon yyyy]		[dd Mon yyyy]

Average velocity = total story points ÷ number of sprints = 37 ÷ 6 ≈ 6.2 story points per sprint.

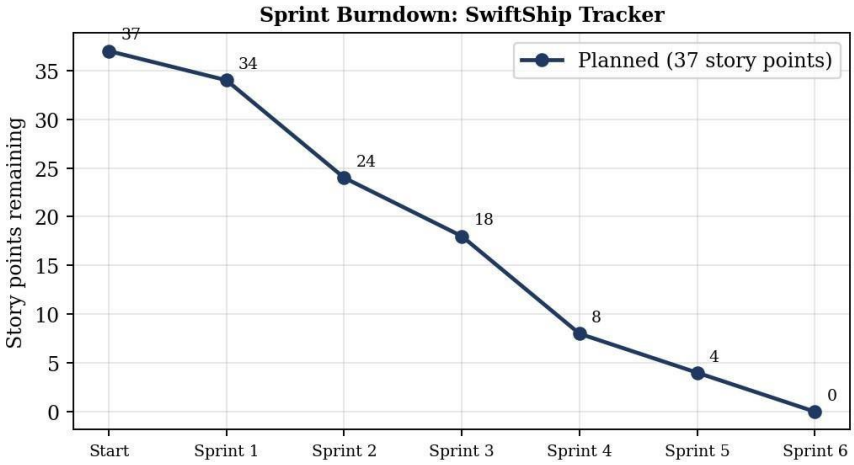


Figure 4: Planned sprint burndown chart

6. PROJECT DEVELOPMENT PHASE

Phase 1: Requirement Analysis and Planning

This phase defined what SwiftShip Tracker must do and how it would be built. The guiding idea was that customers lose time and patience when they must call support or use several apps to book, track, and manage parcels. A single Salesforce platform with a conversational AI agent can remove that friction.

Key Requirements

- Gather requirements from customers and delivery agents.
- Define the system scope and the user personas: Customer, Agent, and Admin.
- Design a data model linking Parcel, Sender, Receiver, and Delivery.
- Identify use cases: parcel booking, tracking, and delivery update automation.
- Select tools: Salesforce Flows, Prompt Builder, and Agentforce.

Project Scope

- Build a scalable CRM solution for a parcel delivery business covering bookings, shipments, and delivery operations.
- Support the end-to-end lifecycle: booking, dispatch, tracking, and delivery confirmation.
- Include data model design, automation (Flows, Apex, email alerts), an AI prompt, and reporting dashboards.
- Provide role-based access for admins, delivery agents, and customers through profiles, permission sets, and Experience Cloud.
- Give customers real-time visibility and give agents simple operational tracking through AI-assisted support.

Stakeholder Mapping

Stakeholder	Role in the System
Customers / Senders	Book parcels, track deliveries, and receive updates through the portal or the AI agent.
Stakeholder	**Role in the System**
Delivery Agents	Manage assigned parcels and update delivery status in real time.

Customer Support Team	Handle delivery issues and answer customer queries.
Admin Team	Configure the org, manage roles and security, and monitor performance.
Developer / Integrator	Implement automation, the AI agent, and backend integrations.

Key Functional Needs

- A booking and tracking interface for customers through an Experience site or with agent help.
- Real-time status updates by agents: Booked → In Transit → Out for Delivery → Delivered.
- Automated email notifications at each delivery stage.
- Sender and receiver management with address, contact, and email fields.
- An AI agent that answers parcel status questions through Prompt Builder.
- Reports and dashboards for parcel flow, delivery performance, and agent activity.
- Role-based security through profiles and permission sets.

Salesforce Features Planned

Area	Features
Custom objects	Parcel__c, Delivery__c, Sender__c, Receiver__c
Standard objects	Account, Contact, Case, Task, EmailMessage, User
Automation	Record-triggered, auto-launched, and scheduled Flows; approval processes
Apex	Apex classes for parcel operations; Batch Apex for periodic updates
UI	Lightning App, Dynamic Forms, Lightning Record Pages, Experience Cloud
AI	Agentforce agent and Prompt Builder template that fetch parcel data by Parcel ID
Email	Custom email alerts and templates for booking, delivery, and completion
Security	Profiles, permission sets, role hierarchy, field-level security, sharing rules

Data Model

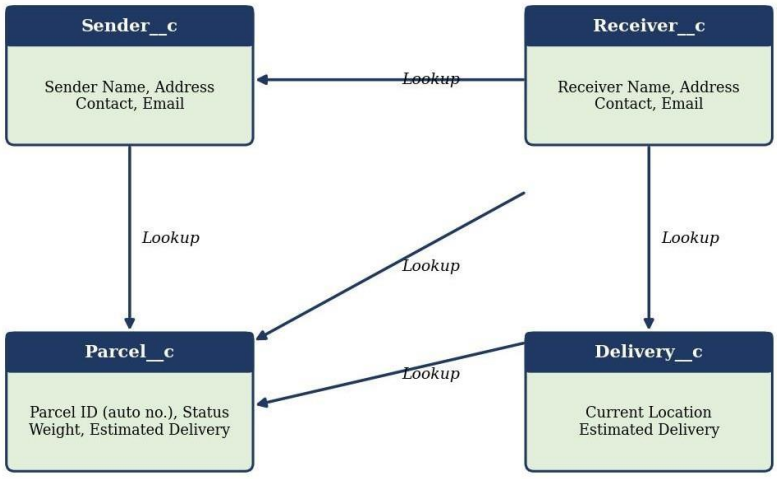


Figure 5: Data model of SwiftShip Tracker

- Parcel__c holds the core parcel details: Parcel ID, Status, Weight, and Estimated Delivery Date, and is linked to Sender and Receiver.
- Delivery__c holds delivery-specific data such as current location and estimated delivery date, linked to its parcel.
- Sender__c stores sender address, contact number, and email, and is linked to many parcels.
- Receiver__c stores receiver address, contact, and email, and is used for delivery reference and notifications.

Security Model Design

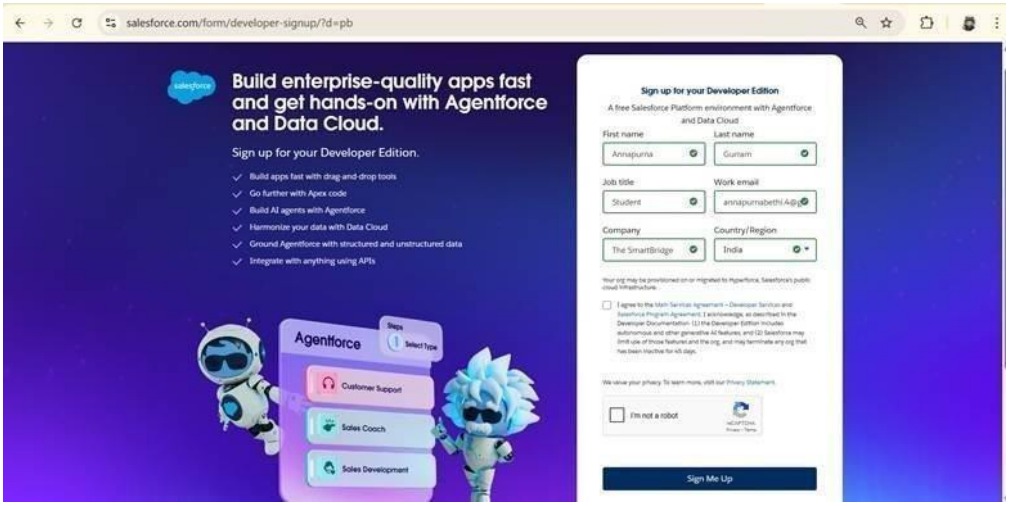
Element	Design
Role hierarchy	Administrator → Delivery Manager → Agent → Customer
Admin profile	Full access to all objects and records.
Delivery Agent profile	Read and edit access to Parcel, Delivery, Sender, and Receiver.
Customer profile	Read-only access to their own parcel details through Experience Cloud.
Support Staff profile	Access to Cases and parcel details for issue resolution.
Permission sets	Extra privileges such as updating delivery status.
Record-level security	Sharing rules on Parcel and Delivery; customers see only records tied to their own sender or receiver details.
Field-level security	Sender email, receiver contact, and location coordinates limited to Admin and Agent profiles.

Phase 2: Backend Development and Configurations

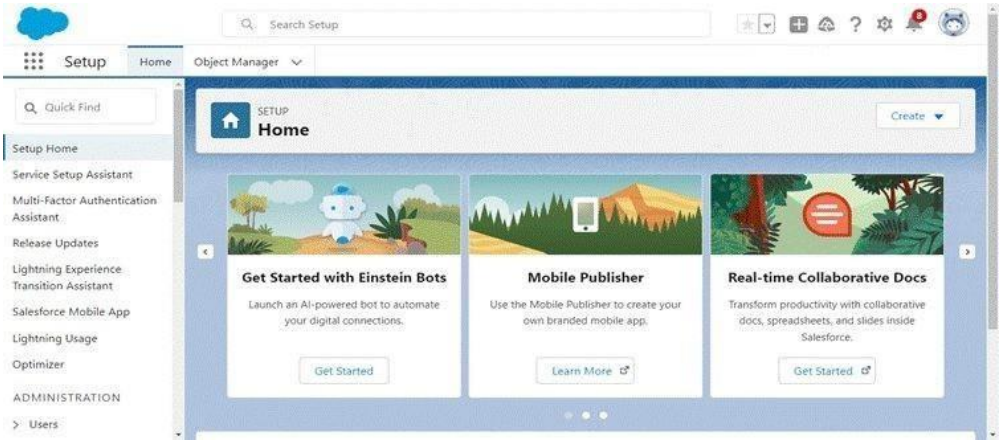
This phase built the working core of the system: the custom objects and their relationships, the navigation tabs, the Prompt Builder template, and the Flow that connects the AI agent to parcel data.

Milestone 1: Salesforce Developer Account

1. Open the Salesforce Developer Edition sign-up page.
2. Enter first and last name, email, job title (Developer), company (college name), and country (India), then submit the form.
3. Open the verification email (it can take from 10 minutes to a couple of hours) and click Verify Account.
4. Set a password, answer the security question, and continue to the Setup home page.



Developer Edition sign-up page

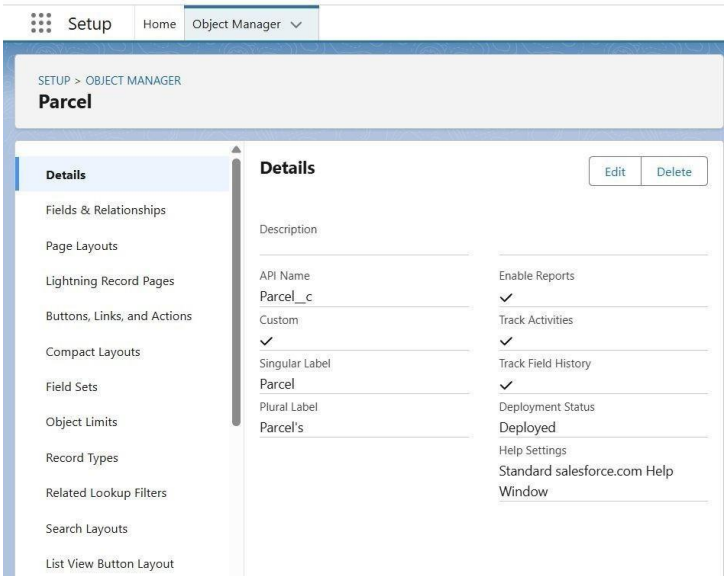


Salesforce Setup home page

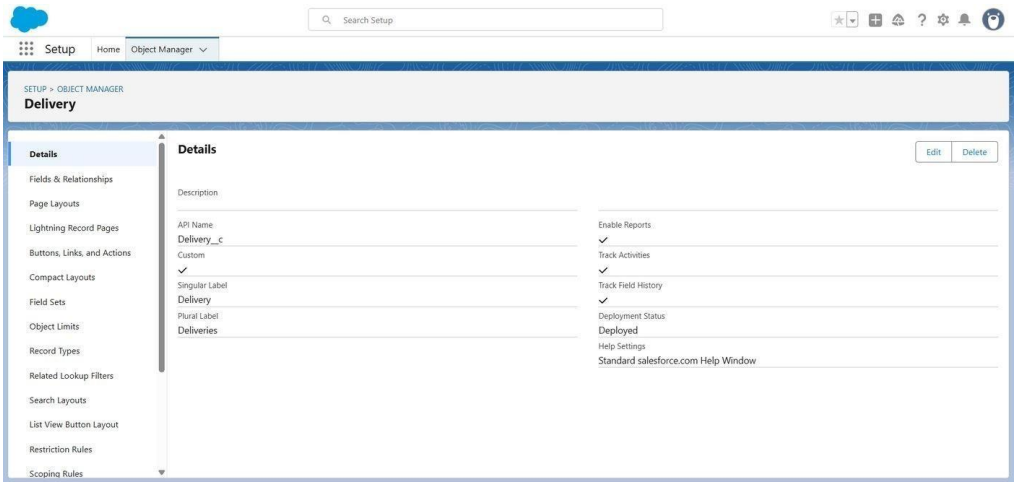
Milestone 2: Custom Objects

An object is a database table that stores organization-specific data. Each of the four objects was created from Setup → Object Manager → Create → Custom Object, with Allow Reports, Allow Search, and Track Field History enabled.

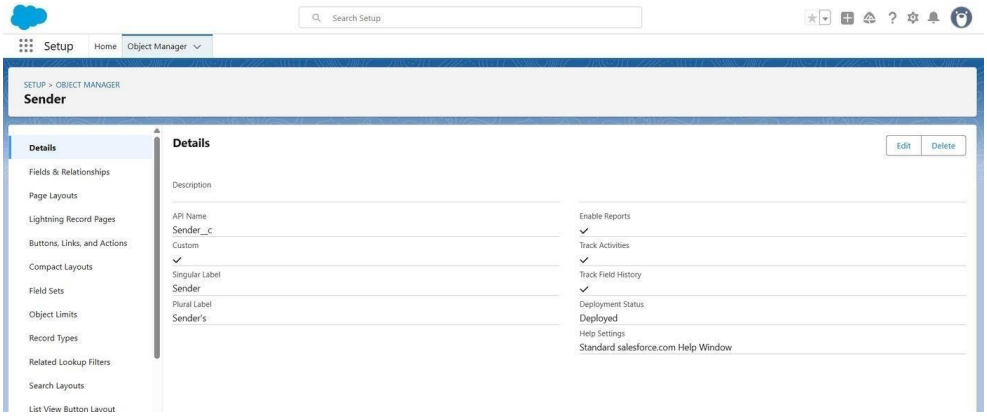
Object	Plural Label	Record Name	API Name
Parcel	Parcels	Parcel Name	Parcel__c
Delivery	Deliveries	Delivery Name	Delivery__c
Sender	Senders	Sender Name	Sender__c
Receiver	Receivers	Receiver Name	Receiver__c



Parcel object details



Delivery object details



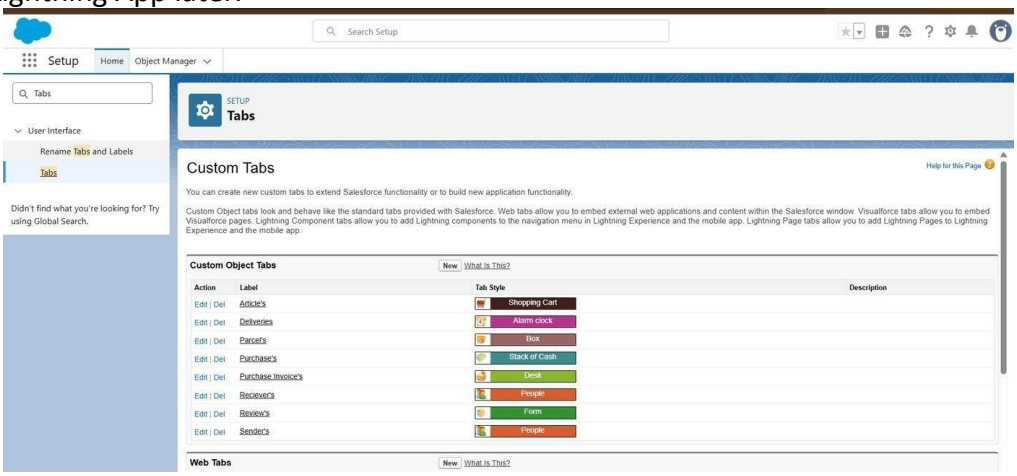
Sender object details



Creating the Receiver object

Milestone 3: Tabs

A custom object tab was created for each object from Setup → Tabs → New under Custom Object Tabs. A tab style was chosen, the Add to Profiles settings were left at their defaults, and "Include Tab" was unchecked on the Add to Custom Apps page so the tabs could be added to the Lightning App later.



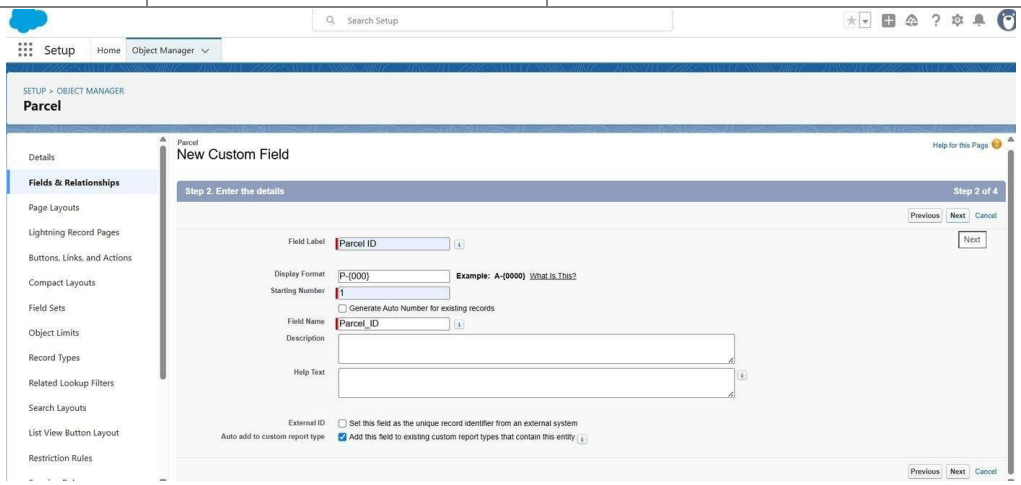
Custom Object Tabs for the four objects

Milestone 4: Fields and Relationships

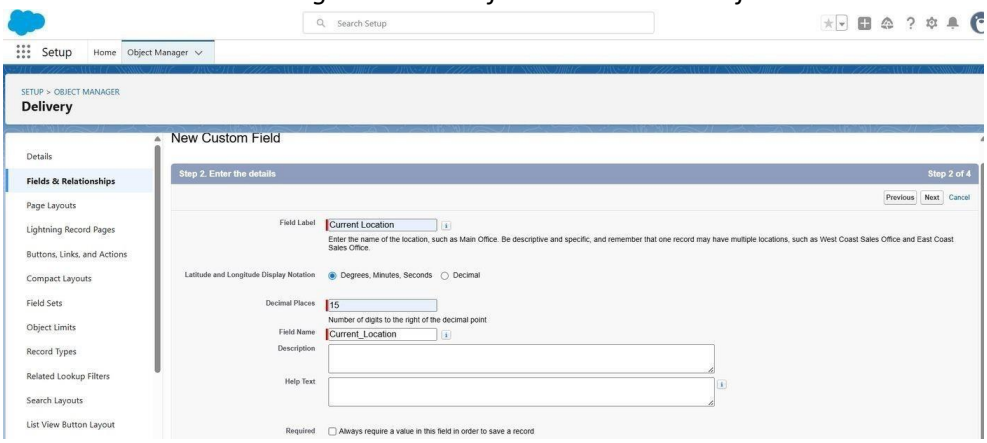
Custom fields and relationships were added to each object from Object Manager → Fields & Relationships → New.

Object	Field	Data Type
Parcel	Parcel ID	Auto Number, format P-{000}
Parcel	Status	Picklist (Booked, In Transit, Out for Delivery, Delivered)
Parcel	Weight	Number
Parcel	Estimated Delivery Date	Date
Parcel	Sender	Lookup to Sender
Delivery	Current Location	Geolocation
Delivery	Estimated Delivery	Date
Object	**Field**	**Data Type**

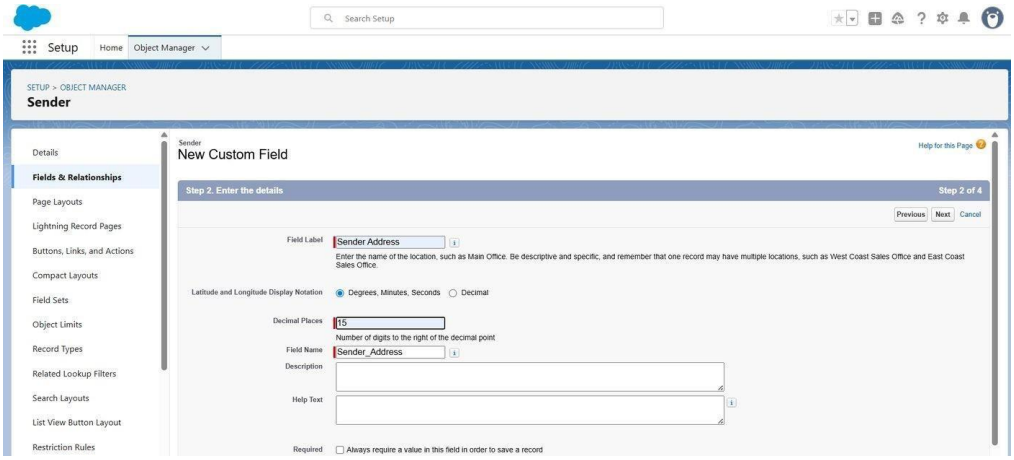
Delivery	Sender	Lookup to Sender
Delivery	Parcel	Lookup to Parcel
Sender	Sender Address	Geolocation
Sender	Sender Contact	Phone
Sender	Sender Email	Email
Receiver	Receiver's Address	Geolocation
Receiver	Receiver Contact	Phone
Receiver	Receiver Email	Email
Receiver	Sender	Lookup to Sender
Receiver	Parcel	Lookup to Parcel



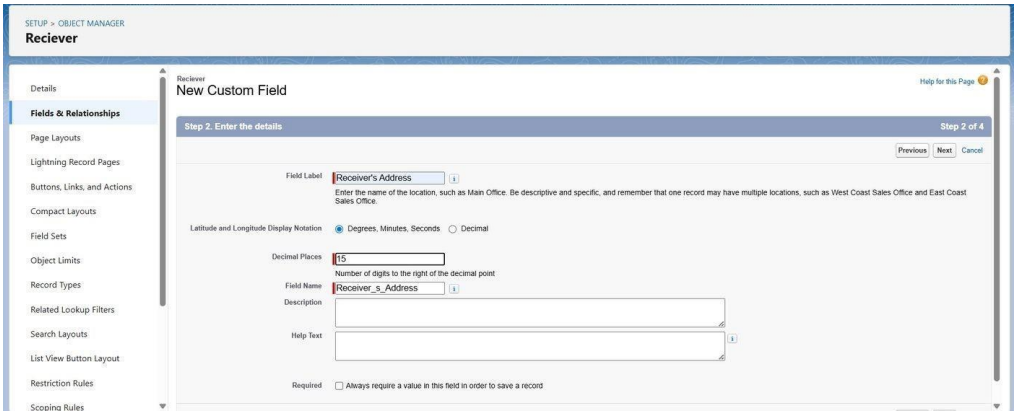
Creating the Parcel ID field on the Parcel object



Creating the Current Location field on the Delivery object



Creating fields on the Sender object



Creating fields on the Receiver object

Milestone 5: Prompt Builder

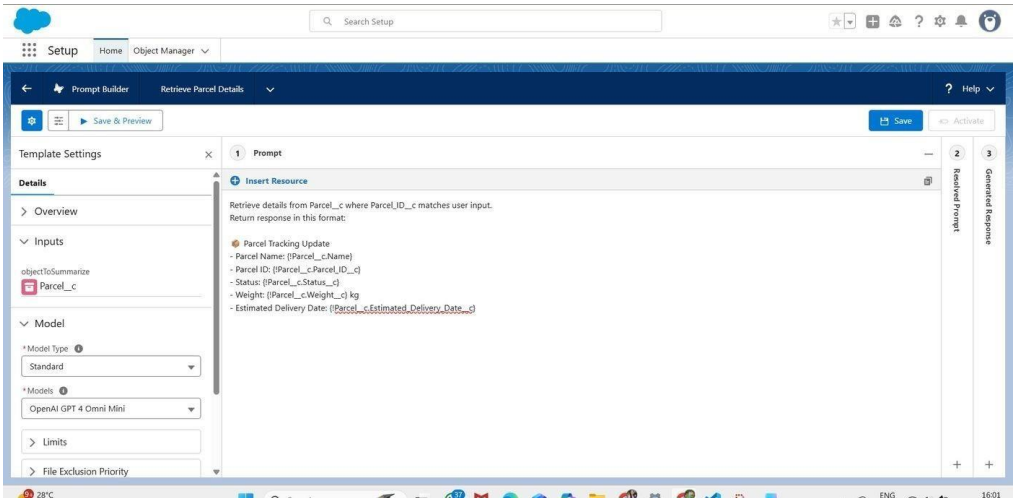
A prompt template lets the Agentforce agent turn a parcel record into a readable reply. It was created from Setup → Prompt Templates → New Prompt Template, using the Record Summary type.

Setting	Value
Label	Retrieve Parcel Details
API Name	Retrieve_Parcel_Details
Description	Retrieves parcel details where Parcel ID matches the user input
Resource	Parcel__c with fields Name, Parcel_ID__c, Status__c, Weight__c, Estimated_Delivery_Date__c

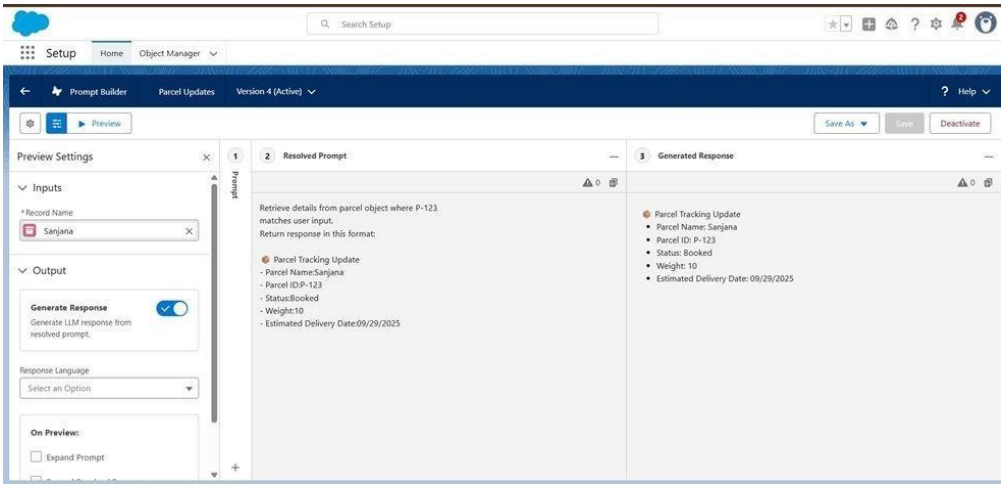
The template text instructs the model to find the Parcel record whose Parcel ID matches the user input and to reply in a fixed tracking format:

```
Parcel Tracking Update
- Parcel Name: {!$Input:Parcel__c.Name}
- Parcel ID: {!$Input:Parcel__c.Parcel_ID__c}
- Status: {!$Input:Parcel__c.Status__c}
- Weight: {!$Input:Parcel__c.Weight__c}
- Estimated Delivery Date: {!$Input:Parcel__c.Estimated_Delivery_Date__c}
```

After the resources were inserted, the template was saved and activated, then tested in the preview by asking the agent to track a sample parcel ID.



Retrieve Parcel Details prompt template with parcel resource fields

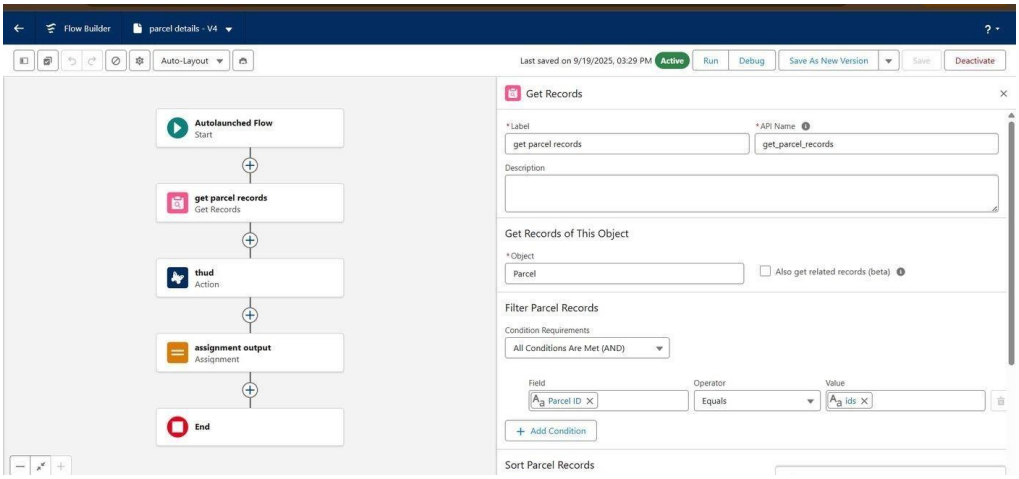


Prompt template preview output

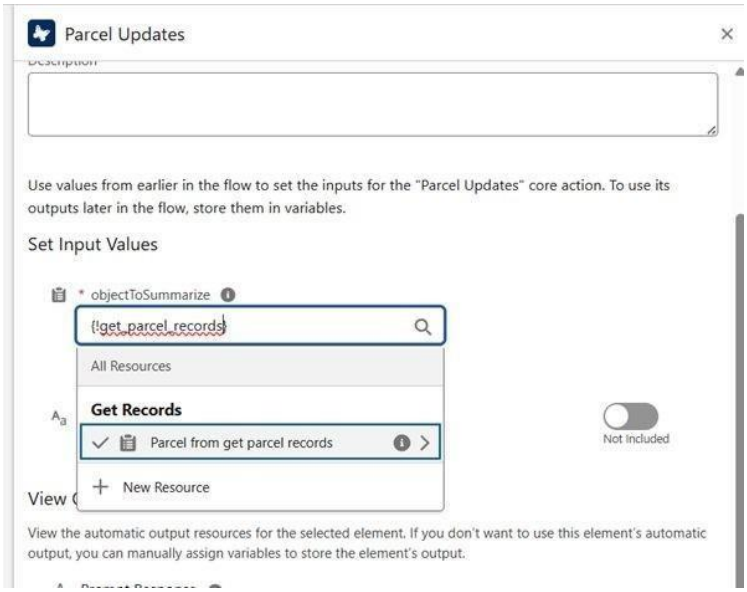
Milestone 6: Flow (Parcel Details)

An Auto-launched Flow (No Trigger) lets the agent pass a Parcel ID in and receive the formatted answer back. It was built in Setup → Flows → New Flow.

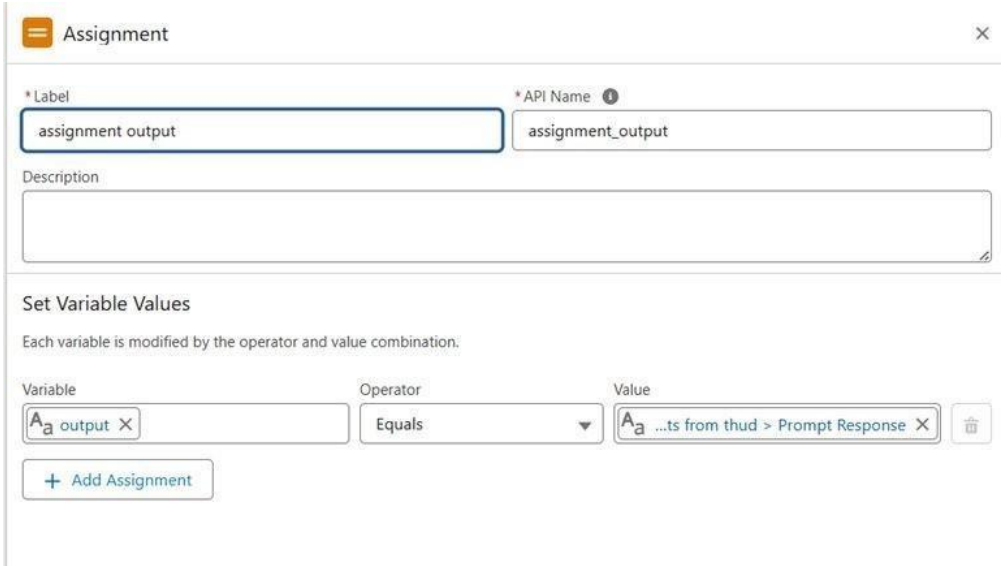
1. Create two variables: Ids (Text, available for input) and Output (Text, available for output).
2. Add a Get Records element labelled Get Parcel Records on the Parcel object, where Parcel_ID__c equals the Ids variable.
3. Add an Action element that calls the Retrieve Parcel Details prompt template, labelled Parcel Updates, with the Parcel returned by Get Records as the object to summarize.
4. Add an Assignment element labelled Assignment Outputs that sets Output equal to the prompt response from the action.
5. Connect Start → Get Records → Action → Assignment → End, then save the flow as Parcel Details and activate it.
6. Open the flow's Edit Access option and grant Flow User Access to the required profiles.



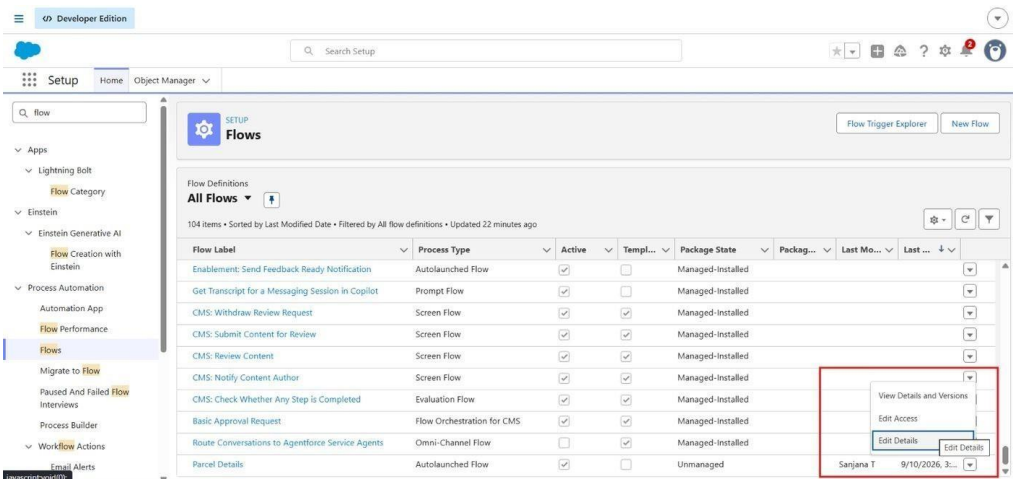
Parcel Details flow: Get Records, Action, Assignment



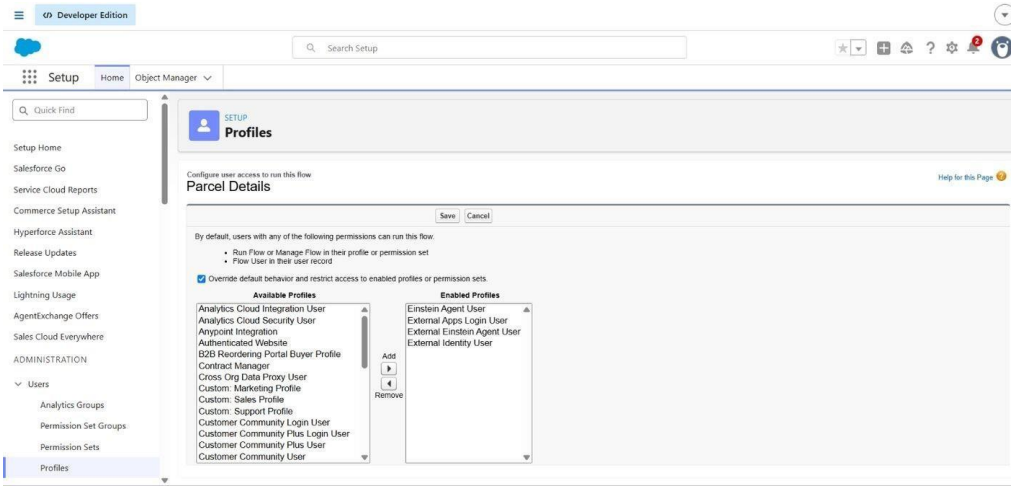
Action element: Parcel Updates



Assignment element: Assignment Outputs



Edit access option for the Parcel Details flow



Flow user access: profiles assigned

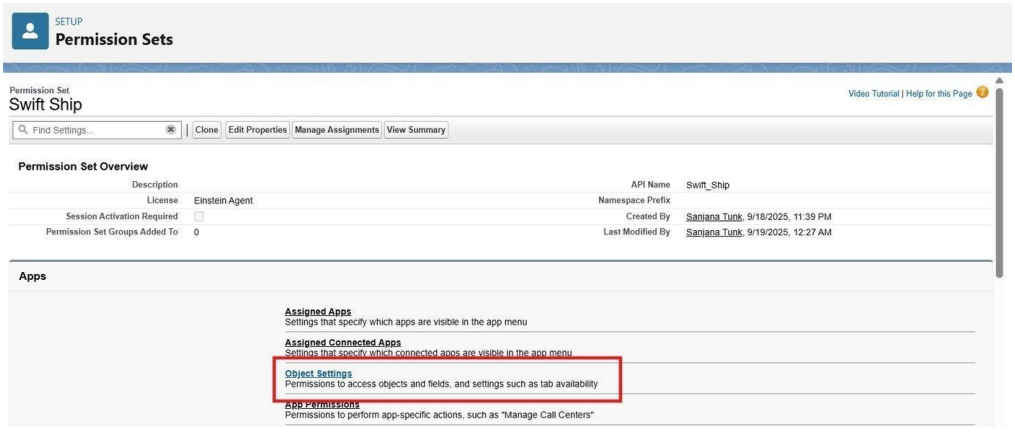
Phase 3: UI/UX Development and Customization

This phase gave agents and the AI agent controlled access to the parcel objects and built the Agentforce agent that customers talk to.

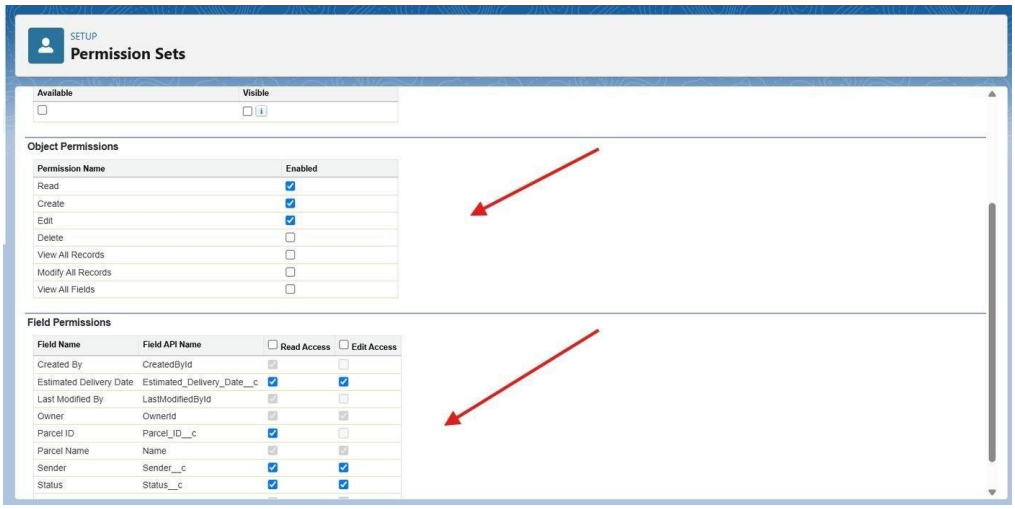
Milestone 7: Permission Set

A permission set named Swift Ship was created from Setup → Permission Sets → New, with the Einstein Agent license. Under Object Settings, access was granted to each of the four objects:

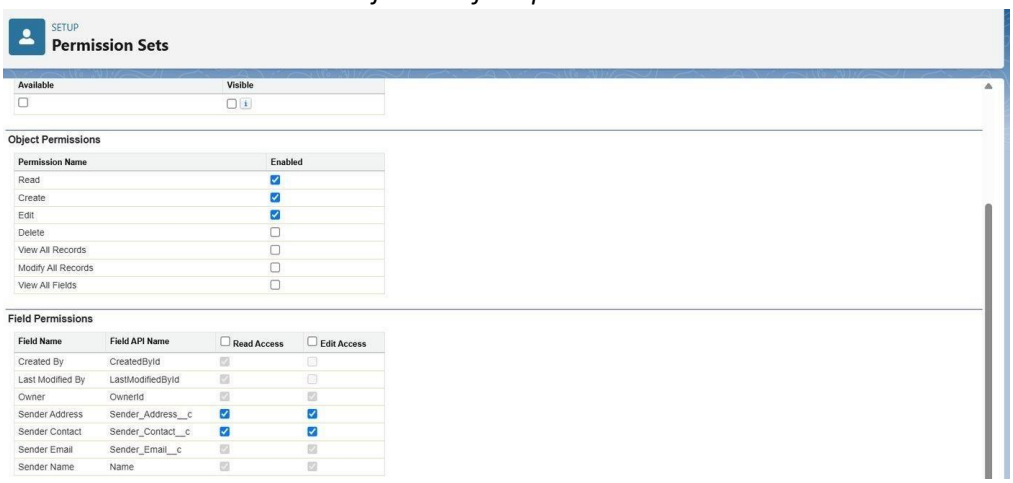
- Parcel__c, Delivery__c, Sender__c, and Receiver__c: Read, Create, and Edit (Delete optional).
- Field permissions: at least Visible on all key fields.



Swift Ship permission set overview



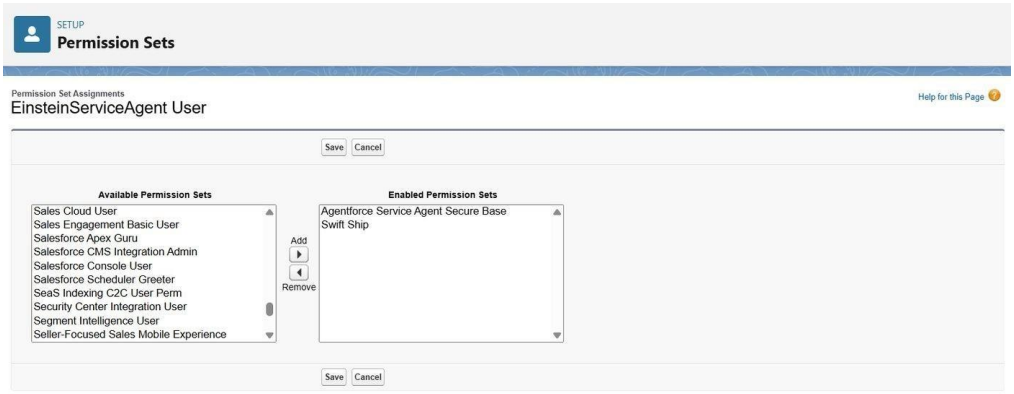
Object and field permissions



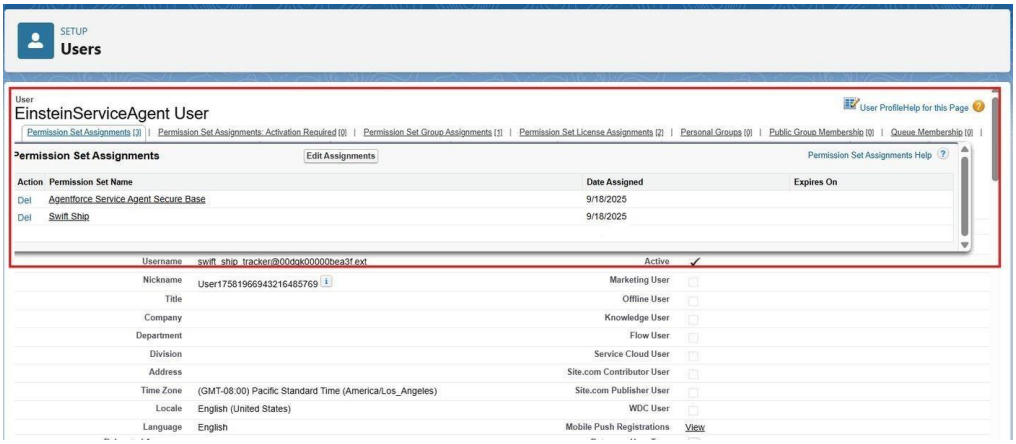
Object and field permissions (continued)

Milestone 8: Assigning the Permission Set to the Agent User

From Setup → Users, the Einstein Service Agent user that represents the agent was opened. In Permission Set Assignments, the Swift Ship permission set was moved to the Enabled list and saved.



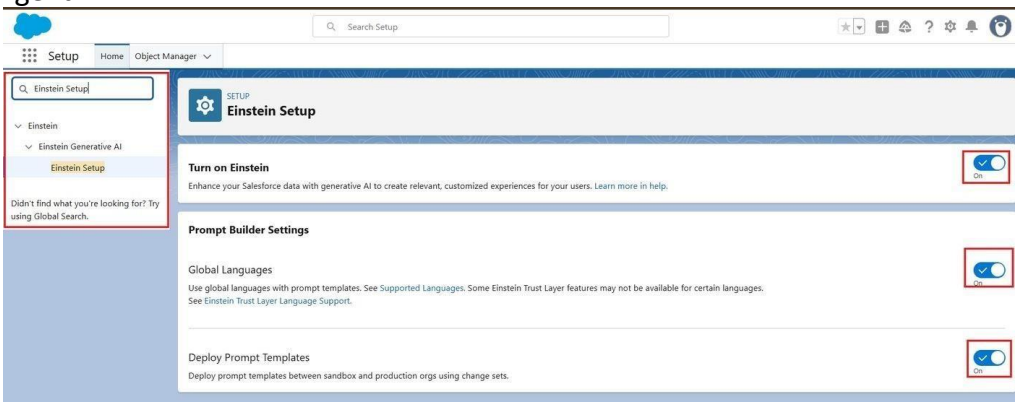
Swift Ship moved to the enabled permission sets



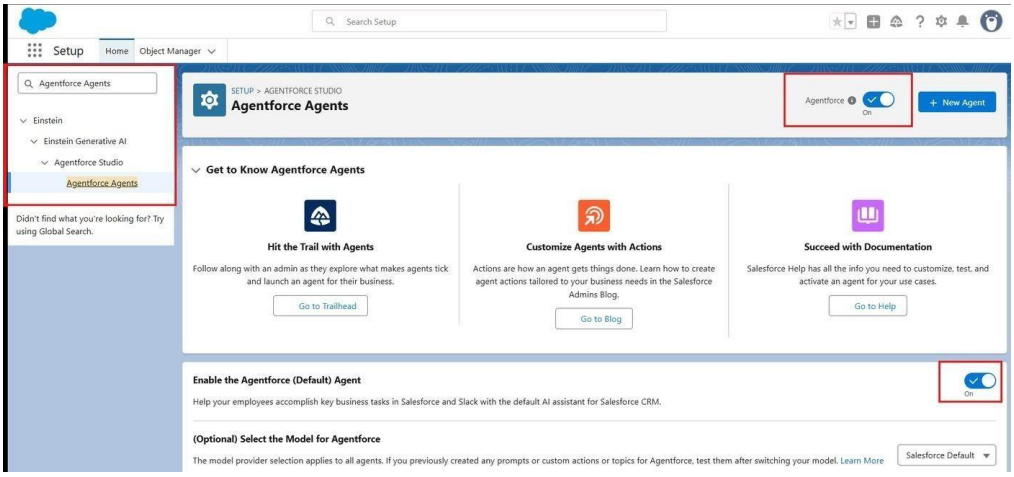
Permission set assignments on the agent user

Milestone 9: Org Setup for Agentforce

- Enabled the default agent from Salesforce Go (Setup → Salesforce Go → Default Agent → Get Started).
- In Einstein Setup, turned on Einstein, Global Languages, and Deploy Prompt Template, then refreshed the page.
- In Setup → Agentforce Agents, turned on Agentforce and the Agentforce Default Agent.



Einstein Setup with Einstein, Global Languages, and Deploy Prompt Template on

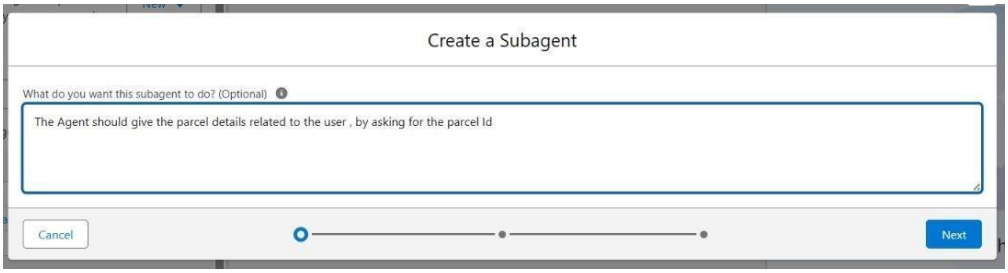


Agentforce and the Default Agent turned on

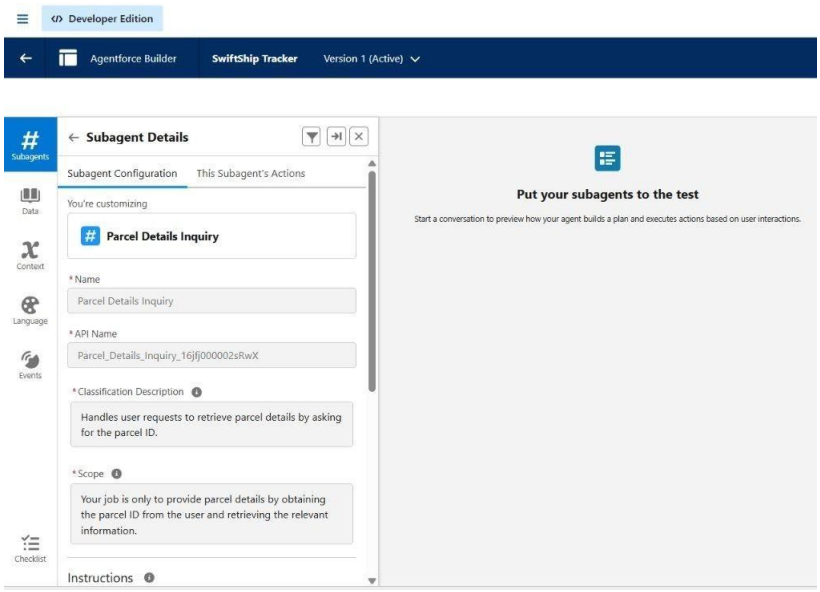
Milestone 10: Building the Agent

1. In Setup → Agentforce Agents, open the default agent in Agent Builder.
2. Create a new subagent for parcel updates and fill in its description.
3. Open the topic actions and create a New Agent Action of type Flow, referencing the Parcel Details flow.

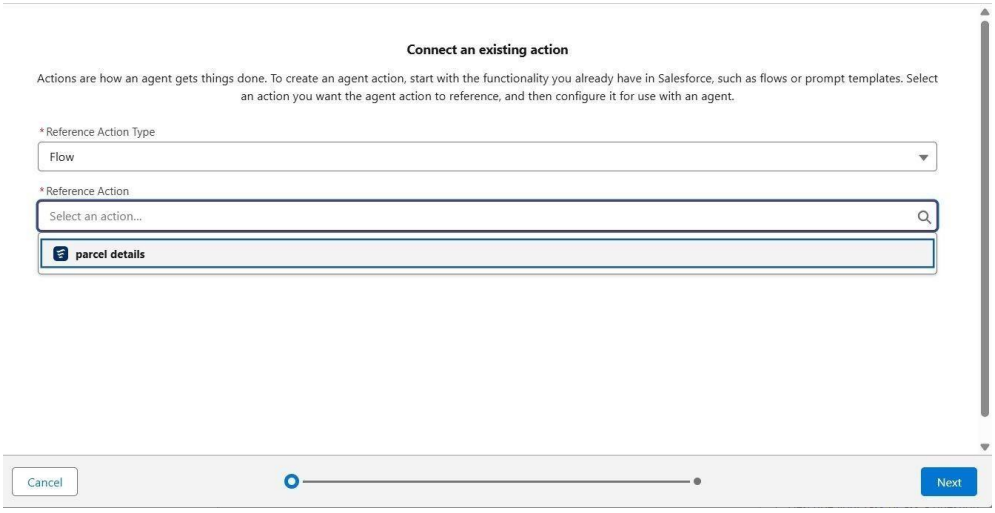
4. Configure the action's inputs and outputs, then add it to the Parcel Updates subagent from the asset library.
5. Open settings and rename the agent to SwiftShip Tracker, save, and activate it.



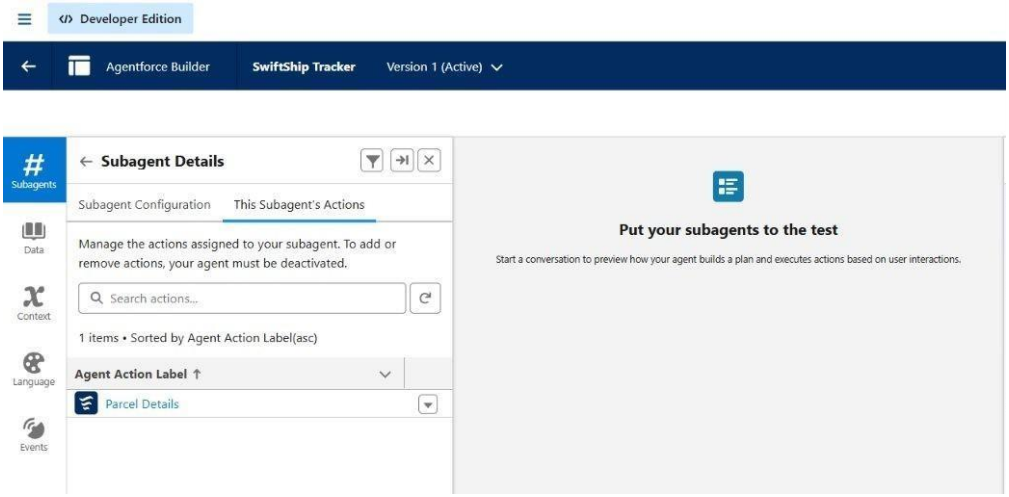
Creating the parcel subagent



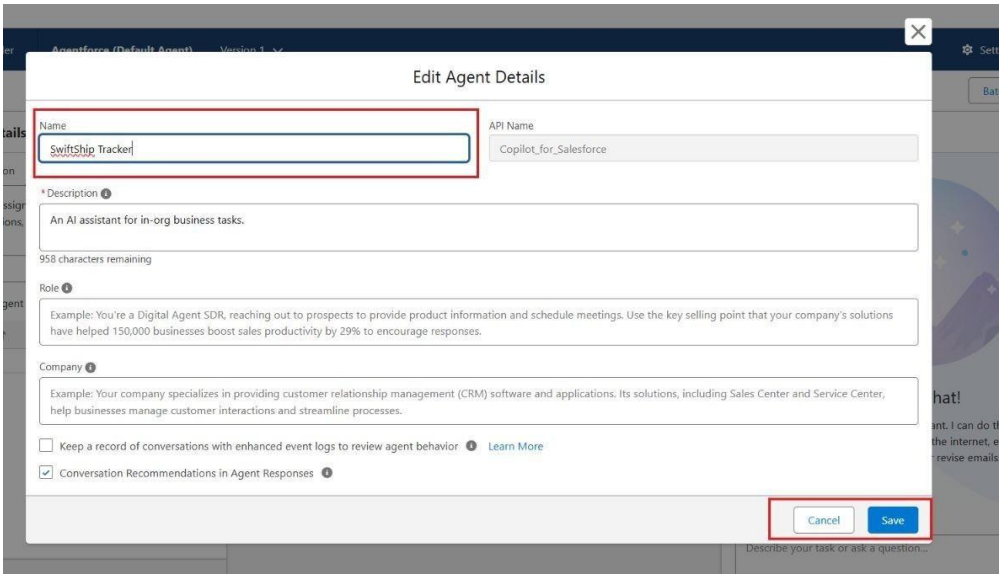
Subagent details in Agent Builder



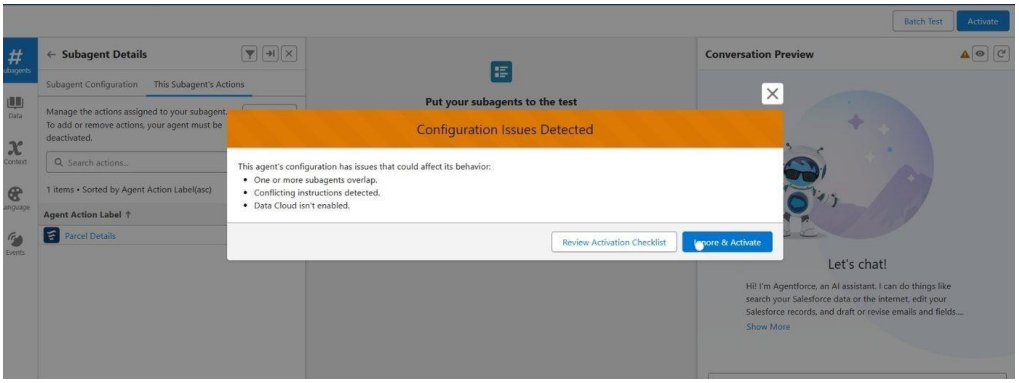
Connecting the Parcel Details flow as an agent action



Flow action listed under the subagent's actions



Renaming the agent to SwiftShip Tracker



Activating the agent

Lightning App, Reports and Dashboards

[If you built the SwiftShip Tracker Lightning App, parcel reports, or dashboard, describe them here with their settings and add screenshots. The guidelines ask for screenshots of reports, so this section should not stay empty.]

[Insert screenshot: Lightning App: SwiftShip Tracker]

[Insert screenshot: Parcel report]

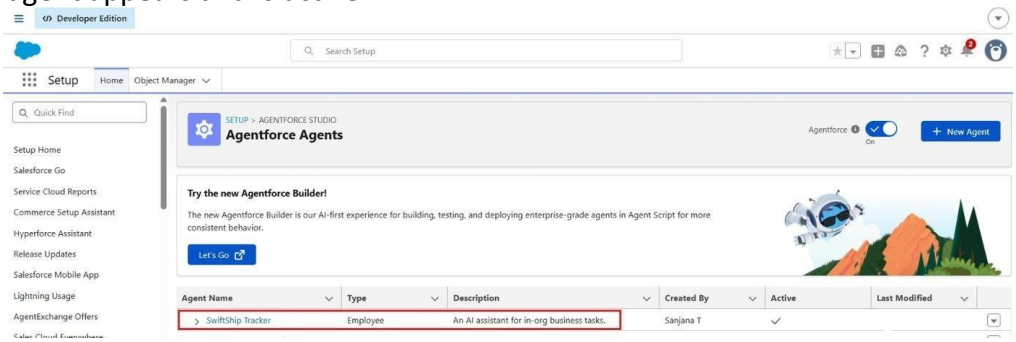
[Insert screenshot: Operations dashboard]

Phase 4: Data Migration, Testing and Security

This phase confirmed that the Parcel Tracking agent recognizes its topic, triggers the assigned Flow, reads parcel data, and works without permission problems. Sample Parcel, Sender, Receiver, and Delivery records were entered before testing.

Verify the Agent

In Setup → Agentforce Agents, the agent list was checked to confirm that the SwiftShip Tracker agent appears and is active.



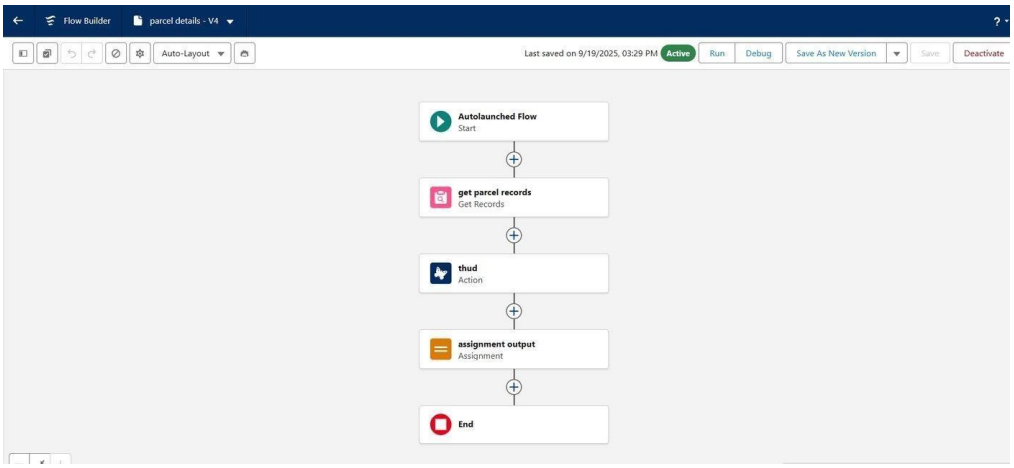
Agentforce Agents list with the SwiftShip Tracker agent

Validate the Assigned Subagent

- In Agent Builder, confirm the Parcel Updates subagent is listed and has a description related to parcel management.
- Confirm the Parcel Details flow action is linked to the subagent.
- Save and confirm the subagent is active.

Check the Flow Association

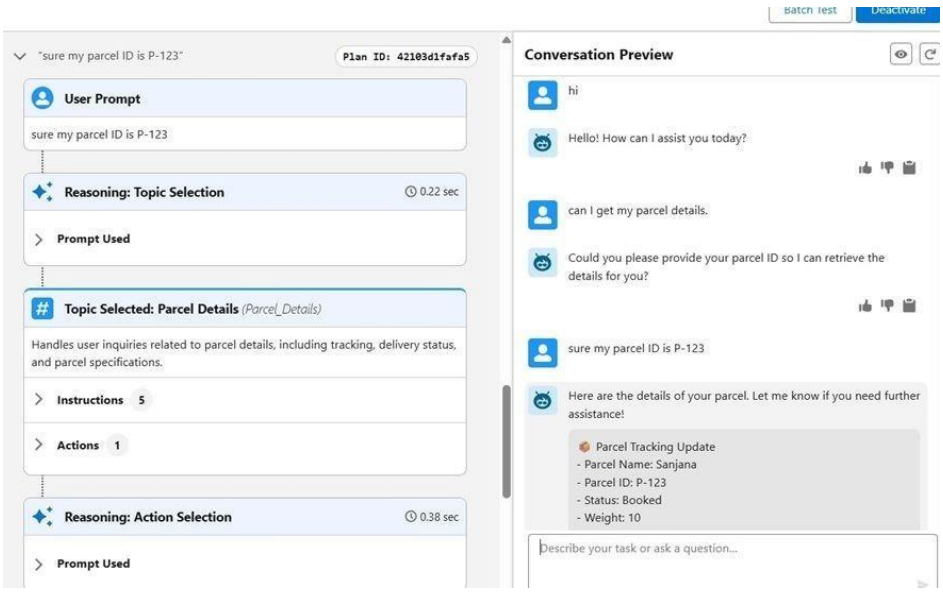
- In Setup → Flows, confirm the Parcel Details flow is active.
- Confirm it handles Parcel ID input, parcel retrieval, and the confirmation message.
- In Advanced Settings, run the flow in System Context Without Sharing when record access is restricted.



Parcel Details flow shown as Active

Activate and Test the Agent

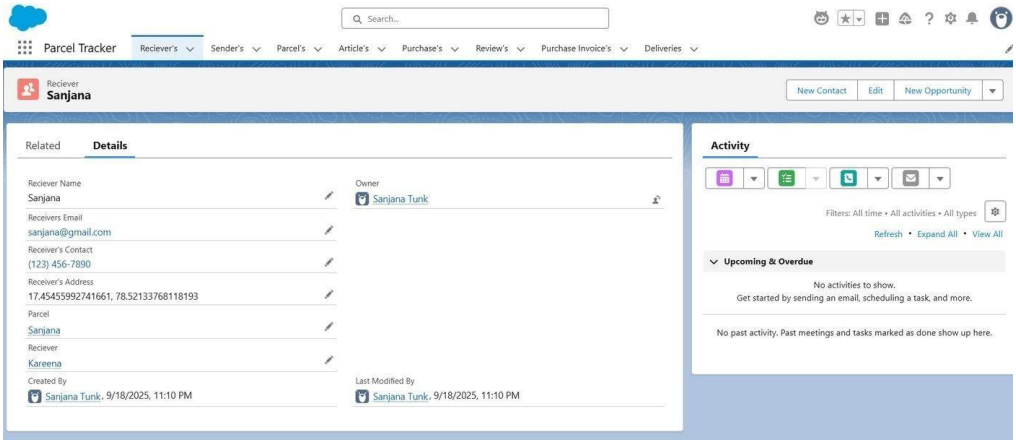
The agent was opened in preview and given queries such as "Track parcel P-001". The reply was compared with the stored Parcel record.



Agent preview returning parcel tracking details

Validate Data in Salesforce

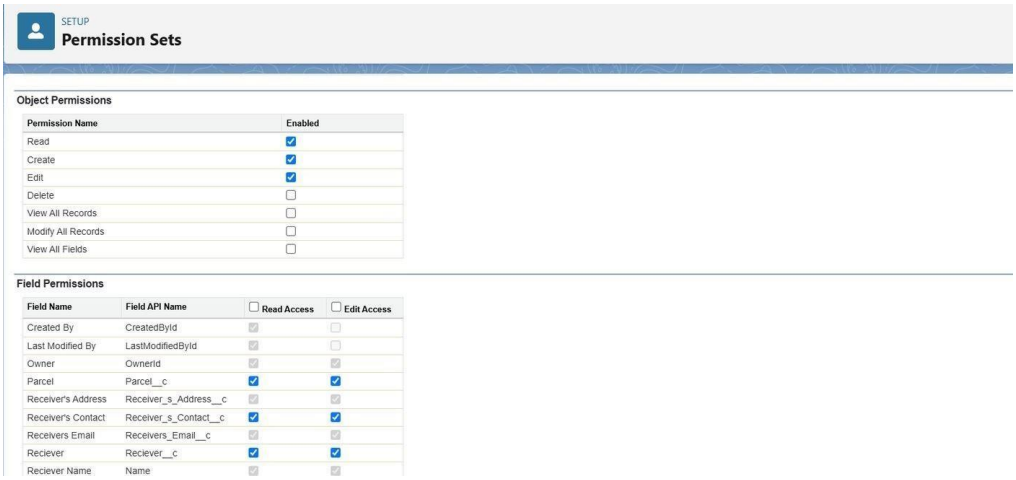
After the agent ran the Flow, the Parcels tab was opened from the App Launcher. The tested Parcel ID was searched and its Status, Weight, and Estimated Delivery Date were compared with the agent's answer.



Parcel record in the Parcel Tracker app

Security Verification

- The Swift Ship permission set was checked for object and field access on all four objects.
- Field-level security was reviewed for sender email, receiver contact, and location fields.
- Profile and role access was checked by logging in as users of each type. [Add the roles you created and tested.]



Field permissions in the Swift Ship permission set

Phase 5: Deployment, Documentation and Maintenance

Deployment Strategy

All configuration and automation was built and tested in a Salesforce Developer Edition org. A Developer Edition org is a standalone environment that is not linked to production, so a live deployment was outside the scope of this academic project. In a real implementation the work would move as follows:

- Build and test in a sandbox environment.
- Package the metadata (objects, fields, flows, prompt templates, agent configuration) and keep it under version control.
- Deploy to production using change sets, or Salesforce DX with a CI/CD pipeline.
- Re-run the test cases in production and confirm that flows, agent behavior, and security work the same way.

Maintenance, Monitoring and Troubleshooting

Regular maintenance keeps parcel, sender, receiver, and delivery data accurate. Flows and agent interactions should be monitored so that failures are caught early. Troubleshooting covers:

- Misrouted agent queries and topic or subagent mismatches.
- Failed Flow executions and permission or access errors.
- Slow or inaccurate Agentforce responses, fixed by refining prompts and subagent instructions.
- Changes in business processes that require Flow or field updates.

Documentation Summary

This document records the purpose, data model, configuration, automation, AI setup, security model, and testing of SwiftShip Tracker. It serves as a blueprint for future developers and administrators and supports training, maintenance, and knowledge transfer.

7. FUNCTIONAL AND PERFORMANCE TESTING

Each feature was tested in the Developer org and screenshots were captured as evidence. Enter the result of each test and attach the screenshot.

ID	Feature Tested	Test Step	Expected Result	Result
T-01	Parcel creation	Create a Parcel record	Record saved; Parcel ID autogenerated (P-001 format)	

T-02	Object relationships	Link a Parcel to a Sender and a Receiver	Related records display correctly	
T-03	Delivery tracking	Create a Delivery with current location and estimated date	Delivery linked to its Parcel	
T-04	Tabs and navigation	Open each object tab	All four tabs open correctly	
T-05	Prompt template	Preview the template with a parcel record	Reply shows ID, status, weight, and estimated date	
T-06	Flow	Run Parcel Details with a valid Parcel ID	Output variable holds the tracking text	
T-07	Agent tracking	Ask the agent: Track parcel P-001	Agent returns correct parcel details	
T-08	Unknown parcel	Ask for a Parcel ID that does not exist	Agent reports that no parcel was found	
T-09	Permissions	Run the agent as the agent user	No access errors on the four objects	
T-10	Security	Log in as each profile	Users see only the permitted objects and fields	

8. ADVANTAGES AND DISADVANTAGES

Advantages

- Centralized parcel, sender, receiver, and delivery information in one platform.
- Better visibility into booking and delivery progress.
- Flows reduce manual effort for common logistics tasks.
- The Agentforce agent gives customers conversational, self-service parcel tracking.
- Prompt Builder keeps tracking replies consistent and data-driven.
- Automated notifications keep customers informed.
- Role-based and field-level security protects personal details.
- The platform can scale to more locations and integrate with external logistics systems.

Disadvantages

- Results depend on accurate and up-to-date parcel data.
- The solution relies on correct Salesforce configuration and automation.
- Agent answers depend on well-written prompts and available data.
- Integrations with external logistics systems add complexity.
- Flows and prompts need maintenance when processes change.
- Users need suitable Salesforce or Experience Cloud access.
- Initial implementation takes significant configuration effort.
- The project was built in a Developer Edition org and has not been deployed to production.

9. CONCLUSION

SwiftShip Tracker shows how Salesforce can be used to build a centralized parcel management and tracking solution. The project combines custom objects and relationships, Flows, a Prompt Builder template, an Agentforce agent, and a role-based security model to support parcel booking, status tracking, and customer communication.

Connecting Agentforce, Prompt Builder, and Flow lets customers get shipment information without always depending on manual support. Building the project covered the full implementation cycle: requirement analysis, data modeling, automation, AI agent configuration, security, testing, and documentation. Although it was built in a Developer Edition org for academic purposes, it follows real-world Salesforce practice and gives a practical foundation for a scalable parcel management solution.

10. FUTURE SCOPE

- Extend the agent to handle more parcel-management queries, such as booking and rescheduling.
- Improve real-time delivery tracking with live location updates.
- Expand customer self-service through Experience Cloud.
- Add advanced delivery-performance analytics.
- Integrate external logistics and courier systems.
- Refine prompts and conversational experiences.
- Move to sandbox testing and CI/CD deployment.
- Automate more parcel operations.

11. APPENDIX

- A. Custom objects: Parcel, Delivery, Sender, Receiver.
- B. Standard objects planned: Account, Contact, Case, Task, EmailMessage, User.
- C. Automation: Parcel Details auto-launched Flow; email alerts.
- D. AI: Agentforce agent SwiftShip Tracker, Prompt Builder template Retrieve Parcel Details.
- E. UI: Lightning App, Lightning Record Pages, Experience Cloud.
- F. Security: Swift Ship permission set, profiles, roles, field-level security, sharing rules.
- G. Analytics: reports and dashboards.
- H. Testing: flow, agent, data retrieval, and security validation.

Thank You